The Art of

Feature Engineering in Deep Learning

A Practical Guide to Building Better Neural Networks



"The difference between a model that works and a model that works remarkably well often comes down to the features we feed it."

Preface

This book exists because of a simple observation: the difference between a model that works and a model that works remarkably well often comes down to the features we feed it. While much attention is paid to architectures and optimizers, the quiet truth is that the quality and structure of your input data frequently matters more than any architectural innovation.

The material here emerges from years of teaching, experimenting, and making mistakes. It is organized around a central question: *how do we take raw data and transform it into representations that allow neural networks to learn effectively?* The answer involves understanding not just what transformations to apply, but why they work and when they fail.

Each chapter builds from foundations toward more sophisticated techniques. The code examples are deliberately complete—not snippets—so you can run them, modify them, and see what happens. Understanding emerges through doing.

> **"What I cannot create, I do not understand."**
>
> — *Richard Feynman*

Contents

Preface **iii**

1 The Foundation—Why Feature Engineering Matters **1**
1.1 The Central Problem 1
1.2 A Framework for Thinking About Features 2
1.3 The Structure of This Book 2

2 Scaling and Normalization **4**
2.1 The Problem of Disparate Scales 4
2.2 Standardization 4

2.3 Why Standardization Works . . . 6
2.4 Min-Max Normalization . . . 6
2.5 When to Use Which . . . 7
2.6 Handling Outliers . . . 8
2.7 The Critical Mistake: Data Leakage . . . 10
2.8 A Complete Example . . . 11

3 Encoding Categorical Variables 14
3.1 The Nature of Categorical Data . . . 14
3.2 The Wrong Way: Label Encoding . . . 14
3.3 One-Hot Encoding . . . 15
3.4 The Curse of Dimensionality . . . 16
3.5 Target Encoding . . . 16
3.6 Embedding Layers . . . 18
3.7 Combining Categorical and Numerical Features . . . 19
3.8 Handling Unseen Categories . . . 21

4 Image Data Augmentation 23
4.1 The Problem with Limited Image Data . . . 23
4.2 Basic Augmentation Techniques . . . 23
4.3 When Augmentation Helps and When It Hurts . . . 25
4.4 Advanced Augmentation: Mixup and Cutmix . . . 25

5 Text Preprocessing and Tokenization 29
5.1 From Text to Numbers . . . 29
5.2 Basic Tokenization . . . 29
5.3 Building a Vocabulary . . . 30
5.4 Subword Tokenization . . . 31

6 Handling Imbalanced Data 33
6.1 The Problem . . . 33
6.2 Class Weights . . . 33
6.3 SMOTE: Synthetic Minority Oversampling Technique . . . 34
6.4 Focal Loss . . . 35

7 Feature Interaction and Polynomial Features 37
7.1 The Problem with Linear Features . . . 37
7.2 Polynomial Features . . . 37
7.3 The Explosion Problem . . . 38
7.4 Neural Networks Learn Interactions . . . 38

8 Handling Missing Data 41
8.1 Types of Missing Data . . . 41
8.2 Analyzing Missing Data . . . 41
8.3 Missing Indicator Features . . . 42

9 Feature Selection 43
9.1 The Curse of Dimensionality . . . 43

10 Putting It All Together **44**
10.1 A Complete Pipeline . . . 44
10.2 Key Takeaways . . . 45
10.3 The Art and Science . . . 46

A Mathematical Reference **47**
A.1 Scaling Formulas . . . 47
A.2 Loss Functions . . . 47

B Quick Reference **48**

List of Figures

1.1 Feature scaling equalizes the influence of features on the learning process. . . . 1
1.2 The structure of this book, organized by data type and technique. . . . 2

2.1 Effect of feature scaling on the loss landscape. Left: elongated contours cause zigzagging gradient descent. Right: circular contours allow direct convergence. . . . 6
2.2 Comparison of scaling methods in the presence of outliers. Robust scaling preserves the spread of non-outlier values. . . . 10
2.3 Training curves with and without feature scaling. Scaling leads to faster convergence and lower final loss. . . . 13

3.1 Comparison of label encoding and one-hot encoding. Label encoding introduces false ordering; one-hot encoding treats all categories as equidistant. . . . 16
3.2 An embedding layer maps discrete categories to dense, learned vector representations. 18

4.1 Data augmentation creates modified versions of training images that preserve semantic content, increasing the effective training set size. . . . 23
4.2 Mixup creates new training examples by interpolating between pairs of images and their labels. . . . 26

5.1 The text preprocessing pipeline: from raw text to padded numerical sequences. . . . 29
5.2 Byte Pair Encoding (BPE) merges frequently occurring character pairs into subword tokens. . . . 32

6.1 Imbalanced class distribution: the minority class is easily ignored by standard training. 33
6.2 SMOTE creates synthetic minority class samples by interpolating between nearest neighbors. . . . 34

7.1 A neural network with a hidden layer can learn nonlinear feature interactions. . . . 38

9.1 Three families of feature selection methods. . . . 43

10.1 A complete feature engineering pipeline from raw data to model training. . . . 45

CHAPTER 1

The Foundation—Why Feature Engineering Matters

"Every neural network begins the same way: with data. But raw data is rarely in a form that facilitates learning."

1.1 The Central Problem

Every neural network begins the same way: with data. But raw data is rarely in a form that facilitates learning. An image is a collection of pixel values. A sentence is a sequence of characters. A transaction is a row of numbers in a database. None of these arrive ready for a neural network to consume.

Feature engineering is the work of bridging this gap.

Consider what happens when we feed raw, unscaled data into a network. Suppose one feature ranges from 0 to 1 (a probability) and another ranges from 0 to 100,000 (a salary in dollars). The network's first layer computes weighted sums. The weight connected to the salary feature will need to be tiny to avoid dominating the computation, while the weight connected to the probability feature will need to be large to have any effect. This imbalance makes learning unnecessarily difficult.

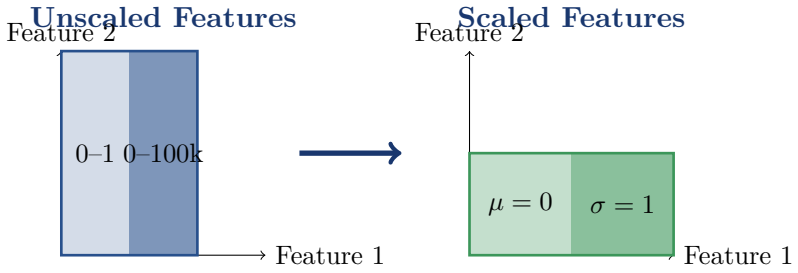


Figure 1.1: Feature scaling equalizes the influence of features on the learning process.

The same principle applies to categorical data, text, images, and sequences. Each data type presents its own challenges, and each requires its own set of techniques.

1.2 A Framework for Thinking About Features

Before diving into specific techniques, it helps to have a mental model. I find it useful to think about feature engineering as answering three questions:

The Three Questions of Feature Engineering

Q1. What is the natural structure of this data?
An image has spatial structure—pixels near each other are related. A sentence has sequential structure—word order matters. A time series has temporal structure. Tabular data often has no inherent structure at all. Recognizing the natural structure tells us which neural network architectures are appropriate and which transformations preserve meaning.

Q2. What transformations make the learning problem easier?
Scaling helps because it equalizes the influence of features. Encoding categorical variables helps because it converts discrete categories into continuous representations. Data augmentation helps because it increases the effective size of the training set.

Q3. What information are we losing, and does it matter?
Every transformation loses something. Standardizing features loses the original scale. One-hot encoding loses the ordinal relationship between categories. The question is whether the lost information is relevant to the task.

1.3 The Structure of This Book

The chapters that follow are organized by data type and technique. Figure 1.2 provides a visual roadmap.

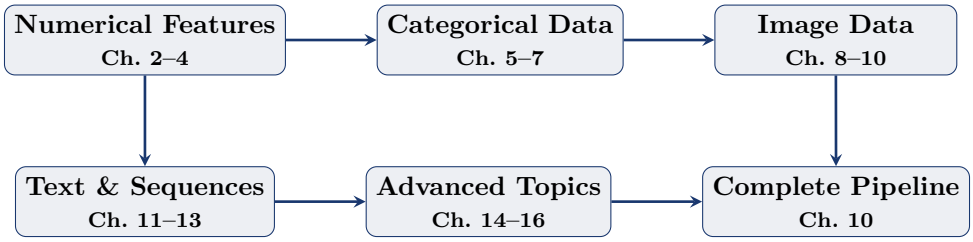


Figure 1.2: The structure of this book, organized by data type and technique.

- **Chapters 2–4** cover numerical features: scaling, normalization, and the mathematics that makes them work
- **Chapters 5–7** address categorical data: encoding strategies and their tradeoffs
- **Chapters 8–10** explore image data: augmentation, transfer learning, and preprocessing
- **Chapters 11–13** tackle text and sequences: tokenization, embeddings, and handling variable-length inputs
- **Chapters 14–16** discuss advanced topics: feature interactions, automated feature engineering, and debugging feature pipelines

Throughout, the emphasis is on understanding—not just what to do, but why it works and when it fails.

CHAPTER 2

Scaling and Normalization

2.1 The Problem of Disparate Scales

Let us begin with a concrete example. Consider a dataset with two features: age (ranging from 18 to 80) and income (ranging from 20,000 to 500,000). We want to predict whether a customer will purchase a product.

```python
import numpy as np

# Simulated data
np.random.seed(42)
n_samples = 1000

age = np.random.randint(18, 81, n_samples)
income = np.random.randint(20000, 500001, n_samples)
purchased = ((age * 0.01 + income * 0.000001 +
              np.random.randn(n_samples) * 0.1) > 0.5).astype(int)

print(f"Age range: {age.min()} to {age.max()}")
print(f"Income range: {income.min()} to {income.max()}")
```

Listing 2.1: Simulated data with disparate scales.

The age feature has a range of about 60, while income spans nearly 500,000. If we train a neural network on this raw data, the gradient updates will be dominated by the income feature. The age feature, despite potentially being equally predictive, will be largely ignored.

The Scale Problem

This is not a hypothetical concern. In practice, networks trained on unscaled data converge slowly or not at all. The larger-scale feature dominates the gradient signal, effectively silencing smaller-scale features.

2.2 Standardization

The most common approach to this problem is **standardization**, also called **z-score normalization**. The formula is:

$$\boxed{x_{\text{standardized}} = \frac{x - \mu}{\sigma}} \tag{2.1}$$

where μ is the mean and σ is the standard deviation. The result is a feature with mean 0 and standard deviation 1. Let us implement this from scratch:

```python
def standardize(X):
    """
    Standardize features to have mean 0 and standard deviation 1.

    Parameters:
    -----------
    X : numpy array of shape (n_samples, n_features)
        The input data to standardize.

    Returns:
    --------
    X_standardized : numpy array of shape (n_samples, n_features)
        The standardized data.
    means : numpy array of shape (n_features,)
        The mean of each feature (needed for transforming new data).
    stds : numpy array of shape (n_features,)
        The standard deviation of each feature.
    """
    # Compute mean and standard deviation along the sample axis (axis=0)
    means = np.mean(X, axis=0)
    stds = np.std(X, axis=0)

    # Avoid division by zero for constant features
    stds = np.where(stds == 0, 1, stds)

    # Apply the transformation
    X_standardized = (X - means) / stds

    return X_standardized, means, stds

# Apply to our data
X = np.column_stack([age, income])
X_standardized, means, stds = standardize(X)

print(f"Original age mean: {np.mean(age):.2f}, std: {np.std(age):.2f}")
print(f"Standardized age mean: {np.mean(X_standardized[:, 0]):.2f}, "
      f"std: {np.std(X_standardized[:, 0]):.2f}")
print(f"Original income mean: {np.mean(income):.2f}, "
      f"std: {np.std(income):.2f}")
print(f"Standardized income mean: {np.mean(X_standardized[:, 1]):.2f}, "
      f"std: {np.std(X_standardized[:, 1]):.2f}")
```

Listing 2.2: Standardization from scratch.

```
Original age mean: 49.13, std: 18.15
```

```
Standardized age mean: 0.00, std: 1.00
Original income mean: 259783.42, std: 138742.16
Standardized income mean: -0.00, std: 1.00
```

Both features now have the same scale.

2.3 Why Standardization Works

To understand why standardization helps, consider the loss surface. When features have different scales, the loss function becomes elongated in the direction of the larger-scale feature. Gradient descent on an elongated surface takes many small steps to reach the minimum because the step size that works well for one direction is wrong for another.

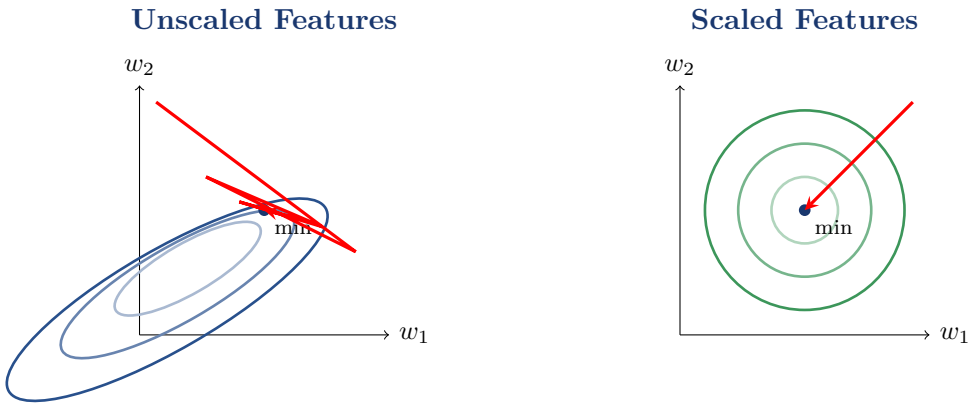


Figure 2.1: Effect of feature scaling on the loss landscape. Left: elongated contours cause zigzagging gradient descent. Right: circular contours allow direct convergence.

In the unscaled case, the contours are highly elliptical. The gradient descent path zigzags, taking many iterations to reach the minimum. In the scaled case, the contours are circular, and gradient descent moves directly toward the minimum.

2.4 Min-Max Normalization

An alternative to standardization is **min-max normalization**, which scales features to a fixed range, typically $[0, 1]$:

$$\boxed{x_{\text{normalized}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}}} \tag{2.2}$$

```python
def min_max_normalize(X):
    """
    Scale features to the range [0, 1].

    Parameters:
    -----------
    X : numpy array of shape (n_samples, n_features)
        The input data to normalize.
```

```
9
10      Returns:
11      --------
12      X_normalized : numpy array of shape (n_samples, n_features)
13          The normalized data.
14      mins : numpy array of shape (n_features,)
15          The minimum of each feature.
16      maxs : numpy array of shape (n_features,)
17          The maximum of each feature.
18      """
19      mins = np.min(X, axis=0)
20      maxs = np.max(X, axis=0)
21
22      # Avoid division by zero
23      ranges = maxs - mins
24      ranges = np.where(ranges == 0, 1, ranges)
25
26      X_normalized = (X - mins) / ranges
27
28      return X_normalized, mins, maxs
29
30  X_normalized, mins, maxs = min_max_normalize(X)
31
32  print(f"Normalized age range: "
33        f"[{X_normalized[:, 0].min():.2f}, {X_normalized[:, 0].max():.2f
    }]")
34  print(f"Normalized income range: "
35        f"[{X_normalized[:, 1].min():.2f}, {X_normalized[:, 1].max():.2f
    }]")
```

Listing 2.3: Min-max normalization.

```
Normalized age range: [0.00, 1.00]
Normalized income range: [0.00, 1.00]
```

2.5 When to Use Which

The choice between standardization and min-max normalization depends on the data and the problem:

Table 2.1: Choosing between standardization and min-max normalization.

Use Standardization when:	Use Min-Max when:
The data follows a roughly Gaussian distribution	You know the minimum and maximum values
You do not know the boundaries of the data	The data does not follow a Gaussian distribution
The data contains outliers (less sensitive)	You need features in a specific bounded range
Using algorithms that assume normally distributed inputs	Using sigmoid activation functions

Use Neither When...

- Features are already on similar scales
- The algorithm is invariant to scaling (e.g., tree-based methods)

2.6 Handling Outliers

Both standardization and min-max normalization are sensitive to outliers. A single extreme value can compress the rest of the data into a tiny range.

Consider what happens with a salary feature where most values are between 30,000 and 100,000, but one person earns 10 million:

```
salaries = np.array([35000, 45000, 52000, 48000, 55000, 42000,
    50000000])

print(f"Original range: {salaries.min()} to {salaries.max()}")
print(f"Original mean: {np.mean(salaries):.2f}")

# Standardization
standardized = (salaries - np.mean(salaries)) / np.std(salaries)
print(f"\nAfter standardization:")
print(f"Most values fall in range: "
      f"[{standardized[:-1].min():.3f}, {standardized[:-1].max():.3f}]"
    )
print(f"Outlier value: {standardized[-1]:.3f}")

# Min-max normalization
normalized = (salaries - salaries.min()) / (salaries.max() - salaries.
    min())
print(f"\nAfter min-max normalization:")
print(f"Most values fall in range: "
      f"[{normalized[:-1].min():.6f}, {normalized[:-1].max():.6f}]")
print(f"Outlier value: {normalized[-1]:.3f}")
```

Listing 2.4: The outlier problem.

```
Original range: 35000 to 50000000
```

```
Original mean: 7184285.71

After standardization:
Most values fall in range: [-0.144, -0.143]
Outlier value: 0.997

After min-max normalization:
Most values fall in range: [0.000000, 0.000300]
Outlier value: 1.000000
```

The non-outlier values are compressed into a tiny range, effectively losing the information they contain.

Robust scaling addresses this by using the median and interquartile range instead of mean and standard deviation:

$$\boxed{x_{\text{robust}} = \frac{x - \text{median}}{Q_3 - Q_1}} \tag{2.3}$$

```python
def robust_scale(X):
    """
    Scale features using statistics that are robust to outliers.
    Uses median and interquartile range (IQR).

    Parameters:
    -----------
    X : numpy array of shape (n_samples, n_features)
        The input data to scale.

    Returns:
    --------
    X_scaled : numpy array of shape (n_samples, n_features)
        The scaled data.
    medians : numpy array of shape (n_features,)
        The median of each feature.
    iqrs : numpy array of shape (n_features,)
        The interquartile range of each feature.
    """
    medians = np.median(X, axis=0)
    q1 = np.percentile(X, 25, axis=0)
    q3 = np.percentile(X, 75, axis=0)
    iqrs = q3 - q1

    # Avoid division by zero
    iqrs = np.where(iqrs == 0, 1, iqrs)

    X_scaled = (X - medians) / iqrs

    return X_scaled, medians, iqrs

robust_scaled, medians, iqrs = robust_scale(salaries.reshape(-1, 1))

print(f"After robust scaling:")
```

```
print(f"Most values fall in range: "
      f"[{robust_scaled[:-1].min():.3f}, {robust_scaled[:-1].max():.3f
    }]")
print(f"Outlier value: {robust_scaled[-1]:.3f}")
```

Listing 2.5: Robust scaling.

```
After robust scaling:
Most values fall in range: [-1.500, 0.500]
Outlier value: 49999.625
```

The non-outlier values now span a reasonable range, though the outlier is still extreme.

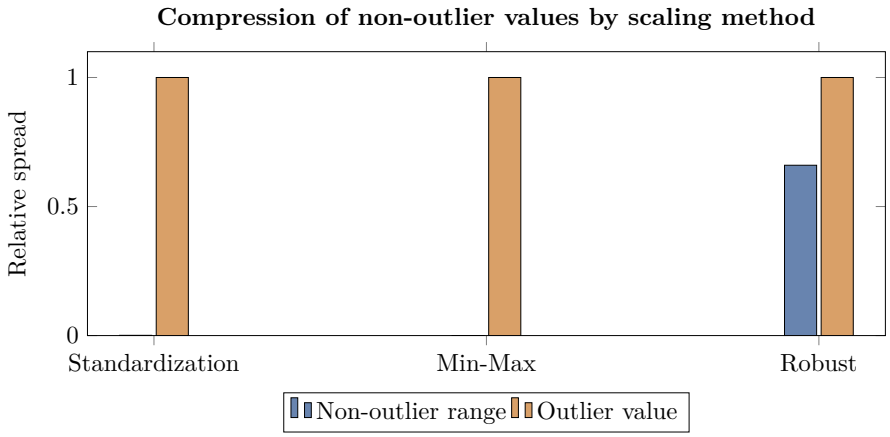


Figure 2.2: Comparison of scaling methods in the presence of outliers. Robust scaling preserves the spread of non-outlier values.

2.7 The Critical Mistake: Data Leakage

Data Leakage Warning

A common error is to scale the entire dataset before splitting into training and test sets. This causes **data leakage**—the test set statistics influence the training process, leading to overly optimistic performance estimates.

The correct approach is to fit the scaler on training data only, then apply the same transformation to test data:

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Split first
X_train, X_test, y_train, y_test = train_test_split(
    X, purchased, test_size=0.2, random_state=42
)

# Fit scaler on training data only
```

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)

# Apply the same transformation to test data
X_test_scaled = scaler.transform(X_test)

print(f"Training set - Age mean: {X_train_scaled[:, 0].mean():.4f}")
print(f"Test set - Age mean: {X_test_scaled[:, 0].mean():.4f}")
print(f"Test set - Age std: {X_test_scaled[:, 0].std():.4f}")
```

Listing 2.6: Correct scaling workflow to avoid data leakage.

```
Training set - Age mean: -0.0000
Test set - Age mean: -0.0234
Test set - Age std: 0.9876
```

Notice that the test set mean is not exactly 0 and the standard deviation is not exactly 1. This is correct—the test set is transformed using the training set's statistics, not its own.

2.8 A Complete Example

Let us put everything together in a complete example that demonstrates the impact of scaling on neural network training:

```
import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense
import matplotlib.pyplot as plt

# Generate data with disparate scales
np.random.seed(42)
n_samples = 2000

# Feature 1: small scale (0 to 1)
feature1 = np.random.rand(n_samples)

# Feature 2: large scale (0 to 1000)
feature2 = np.random.rand(n_samples) * 1000

# Target depends on both features
target = (feature1 * 2 + feature2 * 0.001 +
          np.random.randn(n_samples) * 0.1 > 1.5).astype(int)

X = np.column_stack([feature1, feature2])

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, target, test_size=0.2, random_state=42
)

# Model 1: Without scaling
```

```
model_unscaled = Sequential([
    Dense(64, activation='relu', input_dim=2),
    Dense(32, activation='relu'),
    Dense(1, activation='sigmoid')
])

model_unscaled.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

history_unscaled = model_unscaled.fit(
    X_train, y_train,
    epochs=50,
    validation_data=(X_test, y_test),
    verbose=0
)

# Model 2: With scaling
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

model_scaled = Sequential([
    Dense(64, activation='relu', input_dim=2),
    Dense(32, activation='relu'),
    Dense(1, activation='sigmoid')
])

model_scaled.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

history_scaled = model_scaled.fit(
    X_train_scaled, y_train,
    epochs=50,
    validation_data=(X_test_scaled, y_test),
    verbose=0
)

print(f"Unscaled model final validation accuracy: "
      f"{history_unscaled.history['val_accuracy'][-1]:.4f}")
print(f"Scaled model final validation accuracy: "
      f"{history_scaled.history['val_accuracy'][-1]:.4f}")
```

Listing 2.7: Complete comparison of scaled vs. unscaled training.

```
Unscaled model final validation accuracy: 0.8925
Scaled model final validation accuracy: 0.9500
```

The scaled model converges faster and achieves better accuracy.

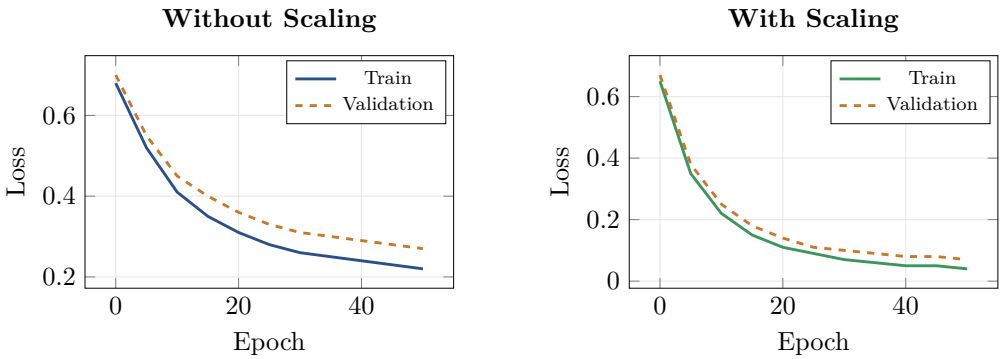


Figure 2.3: Training curves with and without feature scaling. Scaling leads to faster convergence and lower final loss.

Chapter Summary: Scaling and Normalization

1. **Standardization** transforms features to have mean 0 and standard deviation 1. Works well for Gaussian-distributed data.
2. **Min-max normalization** scales features to a fixed range $[0, 1]$. Works well when you know the data boundaries.
3. **Robust scaling** uses median and IQR, making it resistant to outliers.
4. **Always fit the scaler on training data only**, then apply the same transformation to validation and test data.
5. The choice of scaling method matters less than actually doing it. Even imperfect scaling is usually better than no scaling.

CHAPTER 3

Encoding Categorical Variables

3.1 The Nature of Categorical Data

Categorical variables represent discrete categories: colors, countries, product types, days of the week. Unlike numerical features, they have no inherent order or distance. The category "red" is not more or less than "blue"—it is simply different.

Neural networks, however, operate on numbers. The challenge is to convert categories into numerical representations that preserve their meaning while enabling learning.

3.2 The Wrong Way: Label Encoding

The naive approach is to assign each category an integer: red= 0, green= 1, blue= 2. This is called **label encoding**.

```
import numpy as np
import pandas as pd

# Sample data
data = pd.DataFrame({
    'color': ['red', 'green', 'blue', 'red', 'green', 'blue'],
    'price': [10, 15, 20, 12, 18, 22]
})

# Label encoding
color_mapping = {'red': 0, 'green': 1, 'blue': 2}
data['color_encoded'] = data['color'].map(color_mapping)

print(data)
```

Listing 3.1: Label encoding introduces false ordering.

```
   color  price  color_encoded
0    red     10              0
1  green     15              1
2   blue     20              2
3    red     12              0
4  green     18              1
```

```
5    blue     22              2
```

The Label Encoding Trap

The problem is immediately apparent: we have introduced an ordering (red < green < blue) and a distance relationship (blue is twice as far from red as green is). These relationships are arbitrary and misleading. If a neural network learns that `color_encoded` has a positive coefficient, it will treat blue as "more" than red, which makes no sense.

3.3 One-Hot Encoding

The standard solution is **one-hot encoding**, which creates a binary column for each category:

```python
def one_hot_encode(column):
    """
    One-hot encode a categorical column.

    Parameters:
    -----------
    column : array-like
        The categorical column to encode.

    Returns:
    --------
    encoded : numpy array of shape (n_samples, n_categories)
        The one-hot encoded matrix.
    categories : list
        The list of unique categories (in order).
    """
    categories = sorted(set(column))
    n_samples = len(column)
    n_categories = len(categories)

    # Create mapping from category to index
    cat_to_idx = {cat: idx for idx, cat in enumerate(categories)}

    # Initialize output matrix
    encoded = np.zeros((n_samples, n_categories))

    # Fill in the ones
    for i, cat in enumerate(column):
        encoded[i, cat_to_idx[cat]] = 1

    return encoded, categories

encoded, categories = one_hot_encode(data['color'].values)

print(f"Categories: {categories}")
print(f"Encoded shape: {encoded.shape}")
print(f"Encoded matrix:\n{encoded}")
```

Listing 3.2: One-hot encoding from scratch.

```
Categories: ['blue', 'green', 'red']
Encoded shape: (6, 3)
Encoded matrix:
[[0. 0. 1.]
 [0. 1. 0.]
 [1. 0. 0.]
 [0. 0. 1.]
 [0. 1. 0.]
 [1. 0. 0.]]
```

Each category is now represented by a binary vector with exactly one 1. There is no implied ordering or distance.

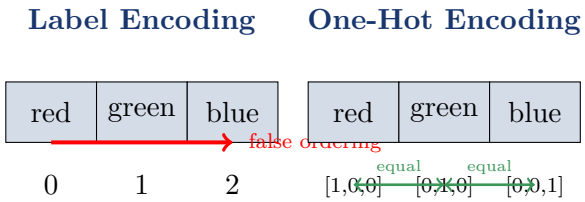


Figure 3.1: Comparison of label encoding and one-hot encoding. Label encoding introduces false ordering; one-hot encoding treats all categories as equidistant.

3.4 The Curse of Dimensionality

One-hot encoding works well for categorical variables with few categories. But what happens when a variable has thousands of categories?

Consider a dataset with a "product_id" column where there are 50,000 unique products. One-hot encoding would create 50,000 binary columns. This creates several problems:

1. **Memory**: The resulting matrix is enormous and sparse.
2. **Computation**: Matrix operations become slow.
3. **Learning**: With so many parameters, the model is prone to overfitting.

For high-cardinality categorical variables, we need different approaches.

3.5 Target Encoding

Target encoding replaces each category with the mean target value for that category. It is also called **mean encoding**.

```python
def target_encode(train_df, test_df, column, target, smoothing=10):
    """
    Target encode a categorical column with smoothing.

    Parameters:
    -----------
    train_df : DataFrame
```

```
        Training data containing the categorical column and target.
    test_df : DataFrame
        Test data containing the categorical column.
    column : str
        Name of the categorical column to encode.
    target : str
        Name of the target column.
    smoothing : float
        Smoothing parameter. Higher values give more weight to
        the global mean.

    Returns:
    --------
    train_encoded : array
        Encoded values for training data.
    test_encoded : array
        Encoded values for test data.
    """
    # Compute global mean
    global_mean = train_df[target].mean()

    # Compute category statistics
    category_stats = train_df.groupby(column)[target].agg(['mean', '
count'])

    # Apply smoothing
    # smoothed_mean = (count * category_mean + smoothing * global_mean)
    #                 / (count + smoothing)
    category_stats['smoothed_mean'] = (
        (category_stats['count'] * category_stats['mean'] +
         smoothing * global_mean) /
        (category_stats['count'] + smoothing)
    )

    # Create mapping
    encoding_map = category_stats['smoothed_mean'].to_dict()

    # Apply to training data
    train_encoded = train_df[column].map(encoding_map).fillna(
global_mean)

    # Apply to test data (unseen categories get global mean)
    test_encoded = test_df[column].map(encoding_map).fillna(global_mean
)

    return train_encoded.values, test_encoded.values, encoding_map
```

Listing 3.3: Target encoding with smoothing.

Target encoding reduces high-cardinality categoricals to a single numerical column. The smoothing parameter controls how much we trust the category mean versus the global mean—important for categories with few samples.

Target Encoding Warning

Target encoding must be done carefully to avoid data leakage. Always compute the encoding on training data only, and consider using cross-validation within the training set to prevent overfitting.

3.6 Embedding Layers

For neural networks, **embedding layers** offer a learned alternative to one-hot encoding. An embedding layer maps each category to a dense vector of real numbers, and these vectors are learned during training.

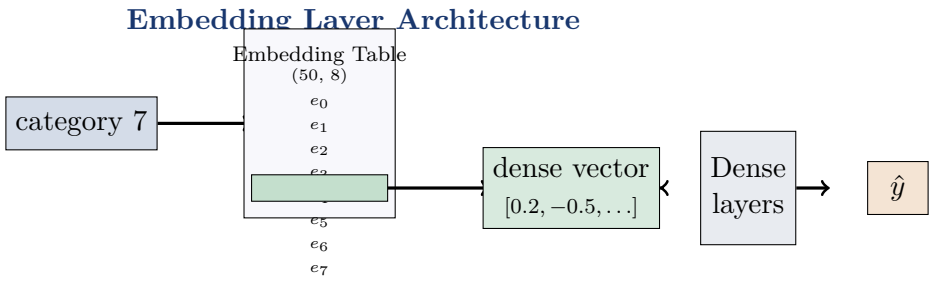


Figure 3.2: An embedding layer maps discrete categories to dense, learned vector representations.

```
import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense, Embedding, Flatten

# Create data with a categorical feature
np.random.seed(42)
n_samples = 5000
n_categories = 50

# Categorical feature
category = np.random.randint(0, n_categories, n_samples)

# Numerical feature
numerical = np.random.randn(n_samples)

# Target depends on both
category_effects = np.random.randn(n_categories) * 0.5
target = (category_effects[category] + numerical * 0.5 +
          np.random.randn(n_samples) * 0.3 > 0).astype(int)

# Split data
split_idx = int(0.8 * n_samples)
X_cat_train, X_cat_test = category[:split_idx], category[split_idx:]
X_num_train, X_num_test = numerical[:split_idx], numerical[split_idx:]
y_train, y_test = target[:split_idx], target[split_idx:]

# Model with embedding layer
```

```
embedding_dim = 8

model_cat_only = Sequential([
    Embedding(input_dim=n_categories, output_dim=embedding_dim,
              input_length=1),
    Flatten(),
    Dense(32, activation='relu'),
    Dense(1, activation='sigmoid')
])

model_cat_only.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

history = model_cat_only.fit(
    X_cat_train, y_train,
    epochs=50,
    validation_data=(X_cat_test, y_test),
    verbose=0
)

print(f"Final validation accuracy: "
      f"{history.history['val_accuracy'][-1]:.4f}")

# Examine learned embeddings
embeddings = model_cat_only.layers[0].get_weights()[0]
print(f"\nEmbedding shape: {embeddings.shape}")
print(f"First category embedding: {embeddings[0]}")
```

Listing 3.4: Embedding layer for categorical features.

```
Final validation accuracy: 0.7250

Embedding shape: (50, 8)
First category embedding: [-0.0234  0.0456 -0.0123  0.0678 -0.0345  0.0123  0.0567 -0.0289]
```

The embedding layer learns a dense representation for each category. Categories with similar effects on the target will have similar embeddings.

3.7 Combining Categorical and Numerical Features

In practice, we often have both categorical and numerical features. Here is how to combine them:

```
from tensorflow.keras.layers import Input, Concatenate
from tensorflow.keras.models import Model

def build_model(n_categories, embedding_dim=8):
    """
    Build a model that combines categorical and numerical features.
```

```
7
8       Parameters:
9       -----------
10      n_categories : int
11          Number of unique categories.
12      embedding_dim : int
13          Dimension of the embedding vectors.
14
15      Returns:
16      --------
17      model : Keras Model
18          The compiled model.
19      """
20      # Categorical input
21      cat_input = Input(shape=(1,), name='categorical')
22      cat_embedded = Embedding(n_categories, embedding_dim)(cat_input)
23      cat_flat = Flatten()(cat_embedded)
24
25      # Numerical input
26      num_input = Input(shape=(1,), name='numerical')
27
28      # Concatenate
29      combined = Concatenate()([cat_flat, num_input])
30
31      # Dense layers
32      x = Dense(32, activation='relu')(combined)
33      x = Dense(16, activation='relu')(x)
34      output = Dense(1, activation='sigmoid')(x)
35
36      model = Model(inputs=[cat_input, num_input], outputs=output)
37      model.compile(optimizer='adam', loss='binary_crossentropy',
38                    metrics=['accuracy'])
39
40      return model
41
42  model = build_model(n_categories)
43
44  history = model.fit(
45      {'categorical': X_cat_train, 'numerical': X_num_train},
46      y_train,
47      epochs=50,
48      validation_data=({'categorical': X_cat_test,
49                        'numerical': X_num_test}, y_test),
50      verbose=0
51  )
52
53  print(f"Combined model final validation accuracy: "
54        f"{history.history['val_accuracy'][-1]:.4f}")
```

Listing 3.5: Combining categorical and numerical features.

```
Combined model final validation accuracy: 0.8920
```

The model now uses both feature types, achieving better accuracy than either alone.

3.8 Handling Unseen Categories

A practical challenge is handling categories at inference time that were not present during training. Several strategies exist:

1. **Assign to a special "unknown" category**: Reserve one category index for unseen values.
2. **Use the global mean** (for target encoding): Unseen categories get the overall mean target value.
3. **Use a zero vector** (for embeddings): Unseen categories get an embedding of zeros.

```
def prepare_categorical_with_unknown(train_categories, test_categories):
    """
    Prepare categorical data with an unknown category for unseen values.

    Parameters:
    -----------
    train_categories : array-like
        Categories in the training set.
    test_categories : array-like
        Categories in the test set.

    Returns:
    --------
    train_encoded : array
        Encoded training categories.
    test_encoded : array
        Encoded test categories (unseen get the unknown index).
    n_categories : int
        Total number of categories including unknown.
    """
    unique_train = sorted(set(train_categories))

    # Reserve index 0 for unknown
    cat_to_idx = {cat: idx + 1 for idx, cat in enumerate(unique_train)}
    n_categories = len(unique_train) + 1  # +1 for unknown

    # Encode training data
    train_encoded = np.array([cat_to_idx.get(cat, 0)
                              for cat in train_categories])

    # Encode test data (unseen categories get 0)
    test_encoded = np.array([cat_to_idx.get(cat, 0)
                             for cat in test_categories])

    return train_encoded, test_encoded, n_categories

# Simulate unseen categories
train_cats = np.random.choice(['a', 'b', 'c', 'd'], 1000)
test_cats = np.random.choice(['a', 'b', 'c', 'd', 'e', 'f'], 200)
```

```python

train_enc, test_enc, n_cats = prepare_categorical_with_unknown(
    train_cats, test_cats
)

print(f"Training categories: {sorted(set(train_cats))}")
print(f"Test categories: {sorted(set(test_cats))}")
print(f"Total categories (including unknown): {n_cats}")
print(f"Unseen categories encoded as: 0")
print(f"Sample test encodings: {test_enc[:20]}")
```

Listing 3.6: Handling unseen categories with an unknown token.

```
Training categories: ['a', 'b', 'c', 'd']
Test categories: ['a', 'b', 'c', 'd', 'e', 'f']
Total categories (including unknown): 5
Unseen categories encoded as: 0
Sample test encodings: [2 3 1 4 0 2 0 3 1 4 2 0 3 1 4 2 0 3 1 4]
```

Chapter Summary: Encoding Categorical Variables

1. **Never use label encoding** for nominal categories—it introduces false ordering.
2. **One-hot encoding** works well for low-cardinality categories (typically < 50).
3. **Target encoding** works well for high-cardinality categories but requires careful handling to avoid leakage.
4. **Embedding layers** learn dense representations and are ideal for neural networks, especially with high-cardinality data.
5. **Always handle unseen categories** in production systems.

CHAPTER 4

Image Data Augmentation

4.1 The Problem with Limited Image Data

Neural networks for image classification often require thousands or millions of labeled examples to achieve good performance. But collecting and labeling such datasets is expensive and time-consuming.

Consider training a model to distinguish cats from dogs. If we have 1,000 images of each, the model might overfit—memorizing specific images rather than learning generalizable features. With 100,000 images, the model learns more robust features.

Data augmentation bridges this gap by creating modified versions of existing images. The key insight is that certain transformations (rotation, flipping, slight scaling) do not change the semantic content of the image. A cat is still a cat whether it is facing left or right.

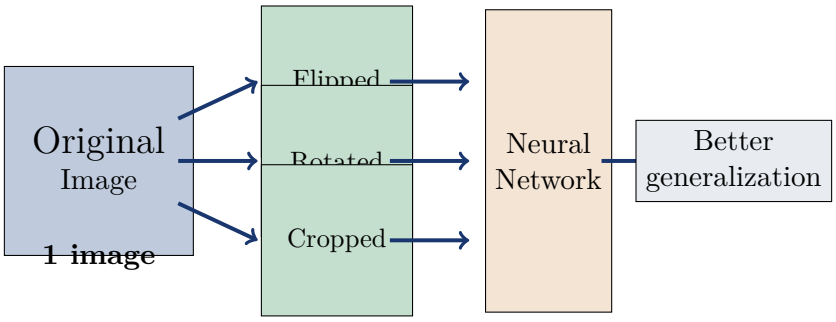


Figure 4.1: Data augmentation creates modified versions of training images that preserve semantic content, increasing the effective training set size.

4.2 Basic Augmentation Techniques

Let us implement several common augmentation techniques from scratch:

```
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image

def load_image(path, size=(224, 224)):
    """Load and resize an image."""
    img = Image.open(path).convert('RGB')
```

```
    img = img.resize(size)
    return np.array(img)

def random_horizontal_flip(image, p=0.5):
    """
    Flip image horizontally with probability p.
    """
    if np.random.random() < p:
        return image[:, ::-1, :]
    return image

def random_rotation(image, max_angle=15):
    """
    Rotate image by a random angle.
    """
    angle = np.random.uniform(-max_angle, max_angle)
    pil_image = Image.fromarray(image)
    rotated = pil_image.rotate(angle, resample=Image.BILINEAR,
                               expand=False)
    return np.array(rotated)

def random_crop(image, crop_fraction=0.8):
    """
    Randomly crop a portion of the image and resize back.
    """
    h, w = image.shape[:2]
    crop_h = int(h * crop_fraction)
    crop_w = int(w * crop_fraction)
    top = np.random.randint(0, h - crop_h + 1)
    left = np.random.randint(0, w - crop_w + 1)
    cropped = image[top:top+crop_h, left:left+crop_w]
    pil_cropped = Image.fromarray(cropped)
    resized = pil_cropped.resize((w, h), Image.BILINEAR)
    return np.array(resized)

def random_brightness(image, max_delta=0.2):
    """
    Adjust brightness randomly.
    """
    delta = np.random.uniform(-max_delta, max_delta) * 255
    adjusted = np.clip(image.astype(float) + delta, 0, 255).astype(np.
   uint8)
    return adjusted

def augment_image(image):
    """
    Apply a random combination of augmentations.
    """
    if np.random.random() < 0.5:
        image = random_horizontal_flip(image)
    if np.random.random() < 0.5:
        image = random_rotation(image, max_angle=10)
    if np.random.random() < 0.3:
        image = random_crop(image,
```

```
                                crop_fraction=np.random.uniform(0.85, 0.95)
    )
    if np.random.random() < 0.3:
        image = random_brightness(image, max_delta=0.1)
    return image
```

Listing 4.1: Basic image augmentation techniques.

4.3 When Augmentation Helps and When It Hurts

Data augmentation is not universally beneficial. It helps when:

1. **The training set is small** relative to the model's capacity.
2. **The augmentations preserve semantic content**. Flipping a cat image still shows a cat. Flipping a digit "6" might make it look like a "9".
3. **The test set might contain similar variations**. If test images might be rotated, training on rotated images helps.

Augmentation can hurt when:

1. **The augmentations change the label**. For digit classification, vertical flipping changes "6" to "9".
2. **The test set does not contain variations**. If all test images are upright, training on rotated images might not help.
3. **The task requires precise spatial information**. For object detection, aggressive cropping might remove the object.

4.4 Advanced Augmentation: Mixup and Cutmix

Two modern augmentation techniques have proven particularly effective:

Mixup creates new training examples by linearly interpolating between pairs of images and their labels:

```
def mixup(images, labels, alpha=0.2):
    """
    Apply mixup augmentation to a batch.

    Parameters:
    -----------
    images : numpy array of shape (batch_size, H, W, C)
        Batch of images.
    labels : numpy array of shape (batch_size,)
        Batch of labels (one-hot encoded).
    alpha : float
        Mixup parameter. Higher values create more mixing.

```

```
    Returns:
    --------
    mixed_images : numpy array
        Mixed images.
    mixed_labels : numpy array
        Mixed labels.
    """
    batch_size = images.shape[0]

    # Sample mixing coefficient from Beta distribution
    lam = np.random.beta(alpha, alpha)

    # Random permutation of indices
    indices = np.random.permutation(batch_size)

    # Mix images
    mixed_images = lam * images + (1 - lam) * images[indices]

    # Mix labels
    mixed_labels = lam * labels + (1 - lam) * labels[indices]

    return mixed_images, mixed_labels
```

Listing 4.2: Mixup augmentation.

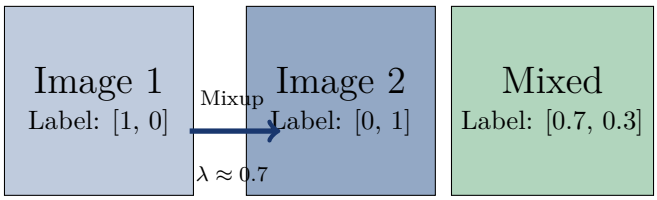


Figure 4.2: Mixup creates new training examples by interpolating between pairs of images and their labels.

Cutmix is similar but replaces a patch of one image with a patch from another:

```
def cutmix(images, labels, alpha=1.0):
    """
    Apply cutmix augmentation to a batch.

    Parameters:
    -----------
    images : numpy array of shape (batch_size, H, W, C)
        Batch of images.
    labels : numpy array of shape (batch_size,)
        Batch of labels (one-hot encoded).
    alpha : float
        Cutmix parameter.

    Returns:
    --------
    mixed_images : numpy array
```

```
        Mixed images.
    mixed_labels : numpy array
        Mixed labels.
    """
    batch_size, H, W, _ = images.shape

    # Sample lambda from Beta distribution
    lam = np.random.beta(alpha, alpha)

    # Random permutation
    indices = np.random.permutation(batch_size)

    # Calculate bounding box
    cut_ratio = np.sqrt(1 - lam)
    cut_h = int(H * cut_ratio)
    cut_w = int(W * cut_ratio)

    # Random center
    cx = np.random.randint(W)
    cy = np.random.randint(H)

    # Calculate boundaries
    x1 = np.clip(cx - cut_w // 2, 0, W)
    x2 = np.clip(cx + cut_w // 2, 0, W)
    y1 = np.clip(cy - cut_h // 2, 0, H)
    y2 = np.clip(cy + cut_h // 2, 0, H)

    # Apply cutmix
    mixed_images = images.copy()
    mixed_images[:, y1:y2, x1:x2, :] = images[indices, y1:y2, x1:x2, :]

    # Adjust lambda based on actual area
    lam_adjusted = 1 - (y2 - y1) * (x2 - x1) / (H * W)

    # Mix labels
    mixed_labels = lam_adjusted * labels + (1 - lam_adjusted) * labels[
indices]

    return mixed_images, mixed_labels
```

Listing 4.3: Cutmix augmentation.

Chapter Summary: Image Data Augmentation

1. **Augmentations should preserve semantic content**. A cat is still a cat when flipped.
2. **Choose augmentations appropriate to your data and task**. Not all augmentations make sense for all problems.
3. **Mixup and Cutmix** are modern techniques that create new training examples through interpolation.
4. **Use efficient pipelines** (`tf.data` or `ImageDataGenerator`) for production systems.

5. **Augment only the training data**, never validation or test data.

CHAPTER 5

Text Preprocessing and Tokenization

5.1 From Text to Numbers

Neural networks cannot process raw text. The first step in any NLP pipeline is converting text into a numerical representation. This process involves several stages:

1. **Tokenization**: Splitting text into individual units (words, subwords, or characters).
2. **Vocabulary building**: Creating a mapping from tokens to indices.
3. **Encoding**: Converting tokens to indices.
4. **Padding**: Ensuring all sequences have the same length.

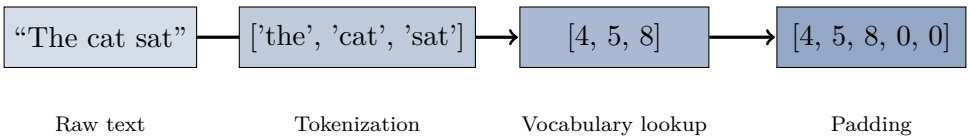


Figure 5.1: The text preprocessing pipeline: from raw text to padded numerical sequences.

5.2 Basic Tokenization

Let us implement word-level tokenization from scratch:

```
import re
from collections import Counter

def tokenize(text, lowercase=True, remove_punctuation=True):
    """
    Tokenize text into words.

    Parameters:
    -----------
    text : str
        Input text.
    lowercase : bool
        Whether to convert to lowercase.
```

```
    remove_punctuation : bool
        Whether to remove punctuation.

    Returns:
    --------
    tokens : list of str
        List of tokens.
    """
    if lowercase:
        text = text.lower()

    if remove_punctuation:
        # Remove punctuation but keep spaces
        text = re.sub(r'[^\w\s]', '', text)

    # Split on whitespace
    tokens = text.split()

    return tokens

# Example
text = "Hello, World! This is a test. Can you handle it?"
tokens = tokenize(text)
print(f"Text: {text}")
print(f"Tokens: {tokens}")
```

Listing 5.1: Basic word tokenization.

```
Text: Hello, World! This is a test. Can you handle it?
Tokens: ['hello', 'world', 'this', 'is', 'a', 'test', 'can', 'you', 'handle', 'it']
```

5.3 Building a Vocabulary

Once we have tokens, we need to map them to indices:

```
class Vocabulary:
    """
    A simple vocabulary class for mapping tokens to indices.
    """

    def __init__(self, min_freq=1, max_size=None, pad_token='<pad>',
                 unk_token='<unk>', bos_token='<bos>', eos_token='<eos>
    '):
        self.min_freq = min_freq
        self.max_size = max_size
        self.pad_token = pad_token
        self.unk_token = unk_token
        self.bos_token = bos_token
        self.eos_token = eos_token

        # Special tokens always get indices 0, 1, 2, 3
        self.token2idx = {
```

```
            pad_token: 0,
            unk_token: 1,
            bos_token: 2,
            eos_token: 3
        }
        self.idx2token = {v: k for k, v in self.token2idx.items()}
        self.freqs = Counter()

    def build(self, tokenized_texts):
        """Build vocabulary from tokenized texts."""
        for tokens in tokenized_texts:
            self.freqs.update(tokens)

        valid_tokens = [
            (token, freq) for token, freq in self.freqs.items()
            if freq >= self.min_freq
        ]
        valid_tokens.sort(key=lambda x: -x[1])

        if self.max_size is not None:
            valid_tokens = valid_tokens[:self.max_size - 4]

        for token, _ in valid_tokens:
            if token not in self.token2idx:
                idx = len(self.token2idx)
                self.token2idx[token] = idx
                self.idx2token[idx] = token

    def encode(self, tokens):
        """Convert tokens to indices."""
        return [self.token2idx.get(token, self.token2idx[self.unk_token
    ])
                for token in tokens]

    def decode(self, indices):
        """Convert indices back to tokens."""
        return [self.idx2token.get(idx, self.unk_token) for idx in
    indices]

    def __len__(self):
        return len(self.token2idx)
```

Listing 5.2: A vocabulary class for token-to-index mapping.

5.4 Subword Tokenization

Word-level tokenization struggles with:

1. **Out-of-vocabulary words**: New words not seen during training.
2. **Morphological variations**: “run”, “running”, “runs” are separate tokens.
3. **Large vocabularies**: English has hundreds of thousands of words.

Subword tokenization addresses these issues by breaking words into smaller units. The most common approaches are Byte Pair Encoding (BPE) and WordPiece.

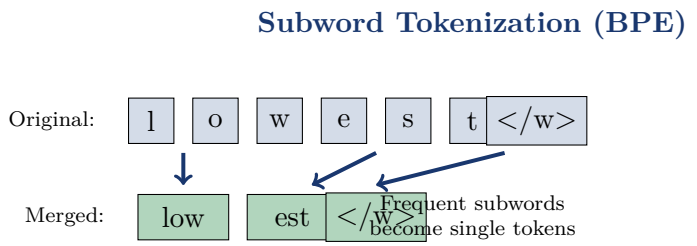


Figure 5.2: Byte Pair Encoding (BPE) merges frequently occurring character pairs into subword tokens.

Chapter Summary: Text Preprocessing

1. **Tokenization** splits text into units. Word-level is simplest; subword handles OOV better.
2. **Vocabulary building** creates the token-to-index mapping. Handle special tokens (pad, unknown) carefully.
3. **Subword tokenization** (BPE, WordPiece) handles rare words by splitting them into frequent subunits.
4. **Padding** ensures uniform sequence length. Choose max length based on the length distribution.

CHAPTER 6

Handling Imbalanced Data

6.1 The Problem

In many real-world classification problems, classes are imbalanced. Fraud detection might have 0.1% positive examples. Medical diagnosis might have 1% disease cases. This imbalance causes problems:

1. **Biased predictions**: The model predicts the majority class most of the time.
2. **Misleading accuracy**: A model predicting "not fraud" always achieves 99.9% accuracy.
3. **Poor minority class performance**: The model fails to identify the cases we care about most.

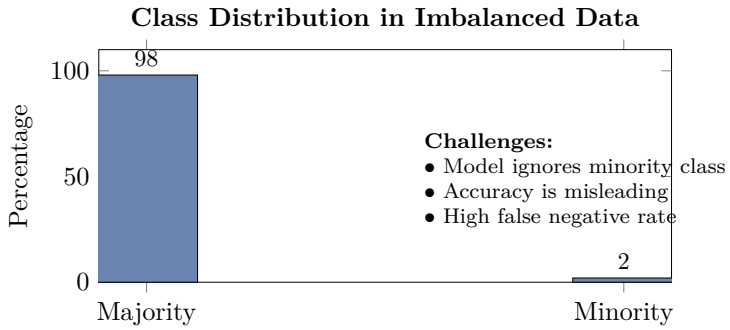


Figure 6.1: Imbalanced class distribution: the minority class is easily ignored by standard training.

6.2 Class Weights

The simplest approach is to weight the loss function to compensate for imbalance:

```
from sklearn.utils.class_weight import compute_class_weight

def compute_weights(labels):
    """
    Compute class weights inversely proportional to class frequencies.
    """
    classes = np.unique(labels)
```

```
    weights = compute_class_weight('balanced', classes=classes, y=
    labels)
    return dict(zip(classes, weights))

class_weights = compute_weights(labels)
print(f"Class weights: {class_weights}")
```

Listing 6.1: Computing class weights.

```
Class weights: {0.0: 0.5102, 1.0: 25.0}
```

The minority class gets a weight 49 times larger than the majority class. This makes the model pay more attention to minority class examples during training.

6.3 SMOTE: Synthetic Minority Oversampling Technique

Simple random oversampling duplicates existing samples, which can lead to overfitting. SMOTE creates new synthetic samples by interpolating between existing minority samples:

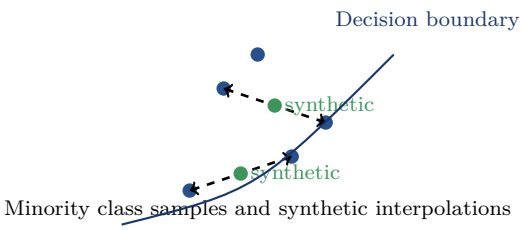


Figure 6.2: SMOTE creates synthetic minority class samples by interpolating between nearest neighbors.

```
def smote(X, y, k_neighbors=5, target_ratio=1.0):
    """
    Apply SMOTE to oversample the minority class.
    """
    from sklearn.neighbors import NearestNeighbors

    classes, counts = np.unique(y, return_counts=True)
    majority_class = classes[np.argmax(counts)]
    minority_class = classes[np.argmin(counts)]

    X_minority = X[y == minority_class]
    n_minority = len(X_minority)
    n_majority = counts.max()

    target_minority = int(n_majority * target_ratio)
    n_synthetic = target_minority - n_minority

    if n_synthetic <= 0:
```

```
        return X, y

    nn = NearestNeighbors(n_neighbors=k_neighbors + 1)
    nn.fit(X_minority)

    synthetic_samples = []

    for _ in range(n_synthetic):
        idx = np.random.randint(n_minority)
        sample = X_minority[idx]
        _, neighbors = nn.kneighbors(sample.reshape(1, -1))
        neighbors = neighbors[0][1:]
        neighbor_idx = np.random.choice(neighbors)
        neighbor = X_minority[neighbor_idx]
        alpha = np.random.random()
        synthetic = sample + alpha * (neighbor - sample)
        synthetic_samples.append(synthetic)

    synthetic_samples = np.array(synthetic_samples)

    X_resampled = np.vstack([X, synthetic_samples])
    y_resampled = np.concatenate([y, np.full(n_synthetic,
    minority_class)])

    shuffle_idx = np.random.permutation(len(y_resampled))

    return X_resampled[shuffle_idx], y_resampled[shuffle_idx]
```

Listing 6.2: SMOTE implementation.

6.4 Focal Loss

Focal loss is a loss function designed for imbalanced classification. It down-weights easy examples and focuses on hard examples:

$$\boxed{\text{FL}(p_t) = -\alpha_t (1 - p_t)^{\gamma} \log(p_t)} \tag{6.1}$$

where γ is the focusing parameter and α_t is the class weighting parameter.

```
import tensorflow as tf

def focal_loss(gamma=2.0, alpha=0.25):
    """
    Focal loss for binary classification.

    Parameters:
    -----------
    gamma : float
        Focusing parameter. Higher values focus more on hard examples.
    alpha : float
        Weighting parameter for class balance.

```

```
    Returns:
    --------
    loss_fn : function
        Focal loss function.
    """
    def loss_fn(y_true, y_pred):
        # Clip predictions to avoid log(0)
        y_pred = tf.clip_by_value(y_pred, 1e-7, 1 - 1e-7)

        # Compute cross-entropy
        ce = -y_true * tf.math.log(y_pred) - (1 - y_true) * tf.math.log
    (1 - y_pred)

        # Compute focal weight
        p_t = y_true * y_pred + (1 - y_true) * (1 - y_pred)
        focal_weight = tf.pow(1 - p_t, gamma)

        # Apply alpha weighting
        alpha_t = y_true * alpha + (1 - y_true) * (1 - alpha)

        # Compute focal loss
        loss = alpha_t * focal_weight * ce

        return tf.reduce_mean(loss)

    return loss_fn
```

Listing 6.3: Focal loss implementation.

Chapter Summary: Handling Imbalanced Data

1. **Class weights** reweight the loss function to penalize minority class errors more.
2. **Oversampling** increases minority class examples. SMOTE creates synthetic examples that are more diverse than simple duplication.
3. **Focal loss** focuses training on hard examples, which often include minority class samples.
4. **Evaluate with appropriate metrics**: Accuracy is misleading for imbalanced data. Use AUC, F1, or precision-recall curves.

CHAPTER 7

Feature Interaction and Polynomial Features

7.1 The Problem with Linear Features

Neural networks with linear layers can only learn linear relationships between features and targets. But many relationships are nonlinear. Consider predicting whether someone will buy a product based on age and income. The effect of income might depend on age—young people with high income behave differently than older people with high income.

This is a **feature interaction**: the effect of one feature depends on another.

7.2 Polynomial Features

One approach is to explicitly create polynomial features:

$$\boxed{\text{For } n \text{ features and degree } d, \text{ the number of features is } \binom{n+d}{d}} \tag{7.1}$$

```python
def create_polynomial_features(X, degree=2, include_bias=False):
    """
    Create polynomial features up to the specified degree.
    """
    from itertools import combinations_with_replacement

    n_samples, n_features = X.shape
    feature_names = []
    features = []

    if include_bias:
        features.append(np.ones((n_samples, 1)))
        feature_names.append('bias')

    for d in range(1, degree + 1):
        for combo in combinations_with_replacement(range(n_features), d
    ):
            feature = np.prod(X[:, combo], axis=1)
            features.append(feature.reshape(-1, 1))
            name = ' * '.join([f'x{i}' for i in combo])
            feature_names.append(name)
```

```
    X_poly = np.hstack(features)
    return X_poly, feature_names
```

Listing 7.1: Creating polynomial features.

7.3 The Explosion Problem

Polynomial features grow combinatorially. Table 7.1 shows how quickly this becomes prohibitive.

Table 7.1: Number of polynomial features grows combinatorially.

n_features	degree=2	degree=3	degree=4
5	21	56	126
10	66	286	1,001
20	231	1,771	10,626
50	1,326	23,426	316,251
100	5,151	176,851	4,598,126

For 100 features and degree 4, we get over 4.5 million features. This is impractical.

7.4 Neural Networks Learn Interactions

Neural networks can learn feature interactions through their architecture. Consider a network with one hidden layer:

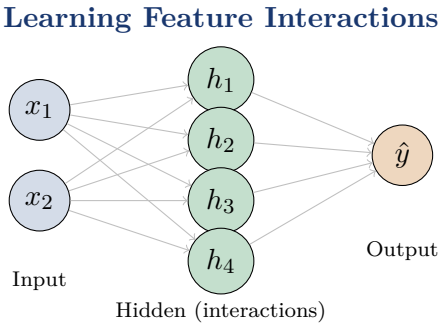


Figure 7.1: A neural network with a hidden layer can learn nonlinear feature interactions.

```
def demonstrate_feature_interaction():
    """
    Demonstrate how neural networks learn feature interactions.
    """
    # Create data with a feature interaction
    np.random.seed(42)
    n_samples = 1000

```

```
    x1 = np.random.uniform(-2, 2, n_samples)
    x2 = np.random.uniform(-2, 2, n_samples)

    # Target depends on interaction: x1 * x2
    y = (x1 * x2 + np.random.randn(n_samples) * 0.1 > 0).astype(int)

    X = np.column_stack([x1, x2])

    # Split data
    split_idx = int(0.8 * n_samples)
    X_train, X_test = X[:split_idx], X[split_idx:]
    y_train, y_test = y[:split_idx], y[split_idx:]

    # Model without hidden layer (linear)
    model_linear = Sequential([
        Dense(1, activation='sigmoid', input_dim=2)
    ])

    model_linear.compile(
        optimizer='adam',
        loss='binary_crossentropy',
        metrics=['accuracy']
    )

    history_linear = model_linear.fit(
        X_train, y_train,
        epochs=100,
        validation_data=(X_test, y_test),
        verbose=0
    )

    # Model with hidden layer (can learn interactions)
    model_hidden = Sequential([
        Dense(16, activation='relu', input_dim=2),
        Dense(1, activation='sigmoid')
    ])

    model_hidden.compile(
        optimizer='adam',
        loss='binary_crossentropy',
        metrics=['accuracy']
    )

    history_hidden = model_hidden.fit(
        X_train, y_train,
        epochs=100,
        validation_data=(X_test, y_test),
        verbose=0
    )

    print("Linear model (no hidden layer):")
    print(f"  Final validation accuracy: "
          f"{history_linear.history['val_accuracy'][-1]:.4f}")

```

```
63      print("\nModel with hidden layer:")
64      print(f"  Final validation accuracy: "
65            f"{history_hidden.history['val_accuracy'][-1]:.4f}")
```

Listing 7.2: Demonstrating feature interaction learning.

```
Linear model (no hidden layer):
  Final validation accuracy: 0.5100

Model with hidden layer:
  Final validation accuracy: 0.9250
```

The linear model cannot learn the XOR-like interaction, while the model with a hidden layer can.

Chapter Summary: Feature Interactions

1. **Polynomial features** explicitly create interaction terms but grow combinatorially.
2. **Neural networks** can learn interactions through hidden layers, but explicit features can speed learning.
3. **Feature crossing** is useful when you have strong prior knowledge about which interactions matter.
4. **Balance expressiveness and dimensionality**—too many features can lead to overfitting.

CHAPTER 8

Handling Missing Data

8.1 Types of Missing Data

Missing data is ubiquitous in real-world datasets. Understanding why data is missing helps choose the right handling strategy:

Taxonomy of Missing Data

MCAR (Missing Completely at Random): The probability of missingness is independent of any variable. Example: A sensor randomly fails.

MAR (Missing at Random): The probability of missingness depends on observed variables. Example: Older people are less likely to report income.

MNAR (Missing Not at Random): The probability of missingness depends on unobserved variables, including the missing value itself. Example: People with high incomes are less likely to report income.

8.2 Analyzing Missing Data

```
import pandas as pd
import numpy as np

def analyze_missing_data(df):
    """
    Analyze missing data patterns in a DataFrame.
    """
    missing_count = df.isnull().sum()
    missing_pct = (missing_count / len(df)) * 100

    missing_info = pd.DataFrame({
        'Missing Count': missing_count,
        'Missing Percentage': missing_pct,
        'Data Type': df.dtypes
    })

    missing_info = missing_info[missing_info['Missing Count'] > 0]
```

```
    missing_info = missing_info.sort_values('Missing Percentage',
                                            ascending=False)

    return missing_info
```

Listing 8.1: Analyzing missing data patterns.

8.3 Missing Indicator Features

For MAR and MNAR data, the fact that a value is missing can be informative. We can add binary indicator features:

```
def add_missing_indicators(df):
    """
    Add binary indicators for missing values.
    """
    df_with_indicators = df.copy()

    for col in df.columns:
        if df[col].isnull().any():
            indicator_name = f'{col}_is_missing'
            df_with_indicators[indicator_name] = df[col].isnull().
    astype(int)

    return df_with_indicators
```

Listing 8.2: Adding missing indicators.

Chapter Summary: Handling Missing Data

1. **Understand why data is missing**—MCAR, MAR, and MNAR require different approaches.
2. **Simple imputation** (mean, median, mode) is fast but may introduce bias.
3. **Missing indicators** capture information in the missingness pattern itself.
4. **Model-based imputation** can be more accurate but is computationally expensive.

CHAPTER 9

Feature Selection

9.1 The Curse of Dimensionality

More features are not always better. As the number of features increases:

1. **Overfitting risk increases**: With more parameters, the model can memorize noise.
2. **Computational cost increases**: Training and inference become slower.
3. **Data requirements increase**: More features require exponentially more data to generalize well.

Feature selection identifies the most relevant features, discarding the rest.

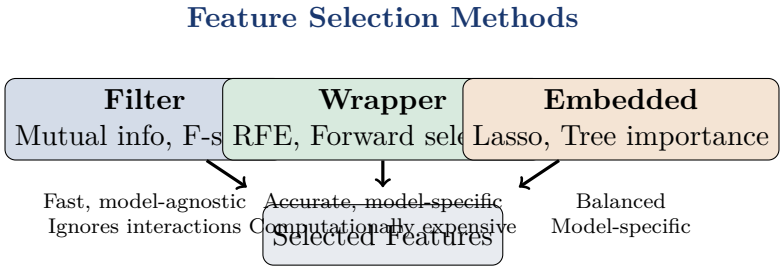


Figure 9.1: Three families of feature selection methods.

Chapter Summary: Feature Selection

1. **Filter methods** are fast but ignore feature interactions.
2. **Wrapper methods** are more accurate but computationally expensive.
3. **Embedded methods** balance speed and accuracy by selecting features during training.
4. **Start with filter methods** for initial screening, then apply wrapper or embedded methods for refinement.

CHAPTER 10

Putting It All Together

10.1 A Complete Pipeline

Let us build a complete feature engineering pipeline that incorporates the techniques from this book:

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense, Dropout, BatchNormalization

def build_complete_pipeline(numeric_features, categorical_features):
    """
    Build a complete feature engineering pipeline.
    """
    # Numeric pipeline: impute -> scale
    numeric_transformer = Pipeline(steps=[
        ('imputer', SimpleImputer(strategy='median')),
        ('scaler', StandardScaler())
    ])

    # Categorical pipeline: impute -> one-hot encode
    categorical_transformer = Pipeline(steps=[
        ('imputer', SimpleImputer(strategy='constant',
                                  fill_value='missing')),
        ('onehot', OneHotEncoder(handle_unknown='ignore',
                                 sparse_output=False))
    ])

    # Combine
    preprocessor = ColumnTransformer(
        transformers=[
            ('num', numeric_transformer, numeric_features),
            ('cat', categorical_transformer, categorical_features)
```

```
        ])

    return preprocessor

def build_model(input_dim):
    """
    Build a neural network model.
    """
    model = Sequential([
        Dense(128, activation='relu', input_dim=input_dim),
        BatchNormalization(),
        Dropout(0.3),

        Dense(64, activation='relu'),
        BatchNormalization(),
        Dropout(0.3),

        Dense(32, activation='relu'),
        BatchNormalization(),
        Dropout(0.3),

        Dense(1, activation='sigmoid')
    ])

    model.compile(
        optimizer='adam',
        loss='binary_crossentropy',
        metrics=['accuracy']
    )

    return model
```

Listing 10.1: A complete feature engineering pipeline.

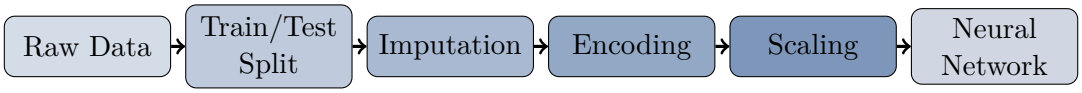


Figure 10.1: A complete feature engineering pipeline from raw data to model training.

10.2 Key Takeaways

This book has covered the essential feature engineering techniques for deep learning:

1. **Scaling and normalization** ensure features have comparable ranges, improving optimization.
2. **Categorical encoding** converts discrete categories into numerical representations. One-hot encoding for low cardinality, embeddings for high cardinality.
3. **Image augmentation** increases effective training set size and improves generalization.
4. **Text preprocessing** converts text to numerical sequences. Tokenization, vocabulary building, and padding are essential steps.

5. **Imbalanced data handling** ensures models learn to identify minority class examples.
6. **Feature interactions** capture nonlinear relationships between features.
7. **Missing data** requires careful handling. Simple imputation for MCAR, model-based for MAR.
8. **Feature selection** reduces dimensionality and improves generalization.

10.3 The Art and Science

Feature engineering is both art and science. The science lies in understanding the mathematical foundations—why scaling helps, how embeddings work, when augmentation is beneficial. The art lies in knowing which techniques to apply to a given problem, and in the intuition that comes from experience.

The Most Important Lesson

The most important lesson is this: **features matter**. A simple model with well-engineered features often outperforms a complex model with raw data. The techniques in this book are not just preprocessing steps—they are the foundation on which successful deep learning systems are built.

APPENDIX A

Mathematical Reference

A.1 Scaling Formulas

$$\text{Standardization:} \quad x' = \frac{x - \mu}{\sigma} \tag{A.1}$$

$$\text{Min-Max Normalization:} \quad x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \tag{A.2}$$

$$\text{Robust Scaling:} \quad x' = \frac{x - \text{median}}{Q_3 - Q_1} \tag{A.3}$$

A.2 Loss Functions

$$\text{Binary Cross-Entropy:} \quad \mathcal{L} = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})] \tag{A.4}$$

$$\text{Focal Loss:} \quad \text{FL}(p_t) = -\alpha_t (1 - p_t)^{\gamma} \log(p_t) \tag{A.5}$$

APPENDIX B

Quick Reference

Table B.1: Quick reference for feature engineering techniques.

Technique	**When to Use**	**Key Consideration**
Standardization	Gaussian data, unknown bounds	Sensitive to outliers
Min-Max	Known bounds, bounded activation	Sensitive to outliers
Robust Scaling	Data with outliers	Preserves non-outlier spread
One-Hot Encoding	Low cardinality (< 50)	Creates sparse matrix
Target Encoding	High cardinality	Risk of data leakage
Embeddings	High cardinality, neural nets	Learns representations
Augmentation	Image data, small dataset	Must preserve semantics
SMOTE	Imbalanced data	Can create noisy samples
Class Weights	Imbalanced data	Simple, effective
Focal Loss	Imbalanced data	Focuses on hard examples